Este trabalho prático tem por finalidade a implementação, em C, de algoritmo para leitura e manipulação de dados formatados de um arquivo texto (csv).

## Formato dos dados:

Nos arquivos ***csv***, cada linha (com exceção da primeira) contém dados relacionados a uma unidade jurisdicional da Justiça Eleitoral. Os dados (colunas) estão separados por vírgula ( ; ), com atributos ordenados conforme especificado no cabeçalho:

```
sigla_tribunal,procedimento,ramo_justica,sigla_grau,uf_oj,municipio_oj,id_ultimo_oj,nome,mesano_cnm1,mesano_sent,casos_novos_2026,julgados_2026,prim_sent2026,suspensos_2026,dessobrestados_2026,cumprimento_meta1,distm2_a,julgm2_a,suspm2_a,cumprimento_meta2a,distm2_ant,julgm2_ant,suspm2_ant,desom2_ant,cumprimento_meta2ant,distm4_a,julgm4_a,suspm4_a,cumprimento_meta4a,distm4_b,julgm4_b,suspm4_b,cumprimento_meta4b
```

O significado de cada variável/coluna é detalhado na tabela a seguir:

| # | Campo | Significado |
|---|---|---|
| **1** | sigla_tribunal | Sigla do tribunal (ex: TJSP, TJES, TRF1) |
| **2** | procedimento | Tipo de procedimento/processo |
| **3** | ramo_justica | Ramo do Judiciário (Estadual, Federal, Trabalho etc.) |
| **4** | sigla_grau | Grau de jurisdição (1º grau, 2º grau) |
| **5** | uf_oj | Unidade da federação do órgão julgador |
| **6** | municipio_oj | Município do órgão julgador |
| **7** | id_ultimo_oj | Identificador do último órgão julgador |
| **8** | nome | Nome do órgão julgador |
| **9** | mesano_cnm1 | Mês/ano de referência dos casos novos da Meta 1 |
| **10** | mesano_sent | Mês/ano da sentença/julgamento |
| **11** | casos_novos_2026 | Processos distribuídos em 2026 |
| **12** | julgados_2026 | Processos julgados em 2026 |
| **13** | prim_sent2026 | Processos com primeira sentença em 2026 |
| **14** | suspensos_2026 | Processos suspensos no período |
| **15** | dessobrestados_2026 | Processos que voltaram a tramitar |
| **16** | cumprimento_meta1 | Indicador de cumprimento da Meta 1 |
| **17** | distm2_a | Processos distribuídos da Meta 2 (ano atual) |
| **18** | julgm2_a | Processos julgados da Meta 2 (ano atual) |
| **19** | suspm2_a | Processos suspensos da Meta 2 (ano atual) |
| **20** | cumprimento_meta2a | Cumprimento da Meta 2 (ano atual) |
| **21** | distm2_ant | Processos antigos elegíveis da Meta 2 |
| **22** | julgm2_ant | Processos julgados do estoque antigo |
| **23** | suspm2_ant | Processos suspensos do estoque antigo |

| | | |
|---|---|---|
| **24** | desom2_ant | Processos dessobrestados do estoque antigo |
| **25** | cumprimento_meta2ant | Cumprimento da Meta 2 (estoque antigo) |
| **26** | distm4_a | Processos distribuídos da Meta 4 (grupo A) |
| **27** | julgm4_a | Processos julgados da Meta 4 (grupo A) |
| **28** | suspm4_a | Processos suspensos da Meta 4 (grupo A) |
| **29** | cumprimento_meta4a | Cumprimento da Meta 4 (grupo A) |
| **30** | distm4_b | Processos distribuídos da Meta 4 (grupo B) |
| **31** | julgm4_b | Processos julgados da Meta 4 (grupo B) |
| **32** | suspm4_b | Processos suspensos da Meta 4 (grupo B) |
| **33** | cumprimento_meta4b | Cumprimento da Meta 4 (grupo B) |

## O que deve ser feito:

O sistema deve ser desenvolvido na linguagem C padrão, a ser compilado e executado no sistema operacional Windows, além de prover interface nas quais devem ser disponibilizadas as seguintes funcionalidades:

1) Concatenar arquivos, com a geração de arquivo "*cvs*" com o conteúdo de todos os arquivos presentes na base de dados disponibilizada para o desenvolvimento deste trabalho;

2) Gerar resumo, em um único arquivo "*csv*" no qual devem contar, para cada "*sigla_tribunal*", as seguintes medidas-resumo: total de julgados_2026, Meta1, Meta2A, Meta2Ant, Meta4A e Meta4B. Para esses campos devem ser usadas as seguintes fórmulas:

| | | |
|---|---|---|
| **1** | $Meta1$ | $\frac{\sum julgados_2026}{\sum casos_novos_2026 + \sum dessobrestados_2026 - \sum suspensos_2026} \times 100$ |
| **2** | $Meta2A$ | $\frac{\sum julgm2_a}{\sum distm2_a - \sum suspm2_a} \times \frac{1000}{7}$ |
| **3** | $Meta2Ant$ | $\frac{\sum julgm2_ant}{\sum distm2_ant - \sum suspm2_ant - \sum desom2_ant} \times 100$ |
| **4** | $Meta4A$ | $\frac{\sum julgm4_a}{\sum distm4_a - \sum suspm4_a} \times 100$ |
| **5** | $Meta4B$ | $\frac{\sum julgm4_b}{\sum distm4_b - \sum suspm4_b} \times 100$ |

3) A partir de município informado pelo usuário, deve ser gerado arquivo "*csv*" com todas as ocorrências relativas a esse município. Por exemplo, caso seja fornecido "**MACAPA**", o arquivo a ser gerado deve ter nome "**MACAPA.csv**" e deve conter todas as linhas da base de dados em que a coluna município_oj seja "**MACAPA**".

## Observações:

- Para armazenar a base de dados, deve ser usado o TAD Lista, como foi tratado em sala de aula.
- O código-fonte deve ser estruturado em ***três arquivos***, conforme explicitado na aula TAD.
- Deve ser implementada 1 (uma) ***função*** para resolver ***cada item*** indicado acima.
- Este trabalho prático pode ser desenvolvido em grupos de ***até 5 integrantes*** e vale 2,0 pontos.
- Caso seja entregue com ***atraso***, o valor total passará a ser, para n dias de atraso, $\mathbf{2,0 \cdot 0,9^n}$.
- O código-fonte deve ser enviado pelo ***AVA***.
- Trabalhos iguais ou com seções iguais/parecidos terão nota 0.
- Na biblioteca ***string.h*** existem funções para manipulação de strings que podem ser úteis.
- Devem ser enviados apenas os arquivos fonte desenvolvidos (não envie arquivos CSV).
- Junto com o código-fonte deve ser postada documentação detalhando o funcionamento do projeto.